SpendWise

- **Team ID :** SWTID-2026-4767

- **Team Leader & Email ID :**

Srinithi R - srinithiramalingam007@gmail.com

- **Team Members & Email ID :**

Kalaimagal P - kalaikalai26193@gmail.com

Swetha R - swethanallu151@gmail.com

Jeeviga V – sanjuvadivel05@gmail.com

Project Title

- SpendWise – A Simple Backend Learning Project using Node.js, Express, MongoDB, and JWT

Project Overview

- SpendWise is a backend application designed to help beginner developers learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

Key Objectives:

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose

- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

Target Users:

- Beginner backend developers
- Students learning Node.js and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

Project Instruction

SpendWise is a backend-only project built using Node.js and Express.

Quick Start:

npm init

npm install

npm run dev

Steps to Create the Project:

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using `npm init`.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using `npm run dev`.
- Test APIs using Postman or Thunder Client at `http://localhost:5000`.

Pre-requisites:

Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code or similar

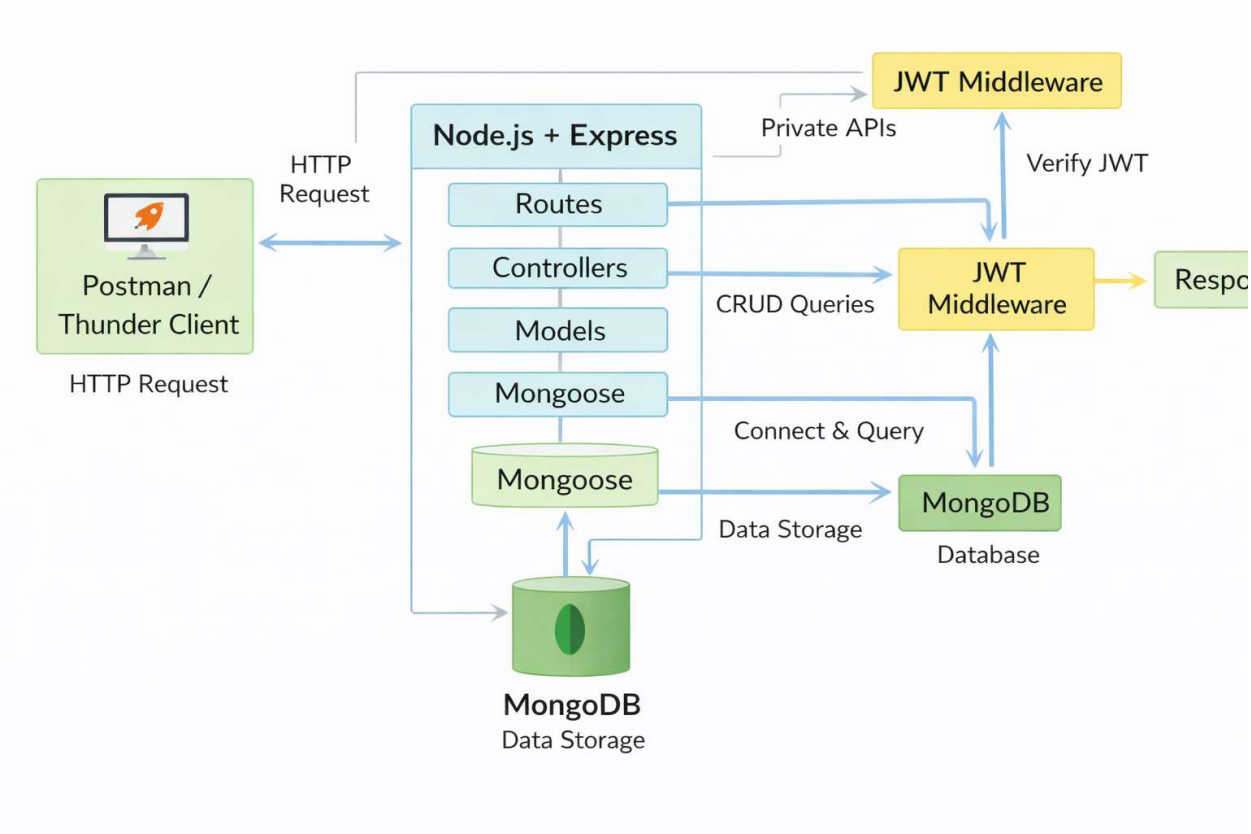
Knowledge Requirements

- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics

Thumbnail for Project

- A simple backend project thumbnail representing:
  Node.js + Express + MongoDB + JWT + CRUD APIs

Project Architecture Image



Technical Architecture

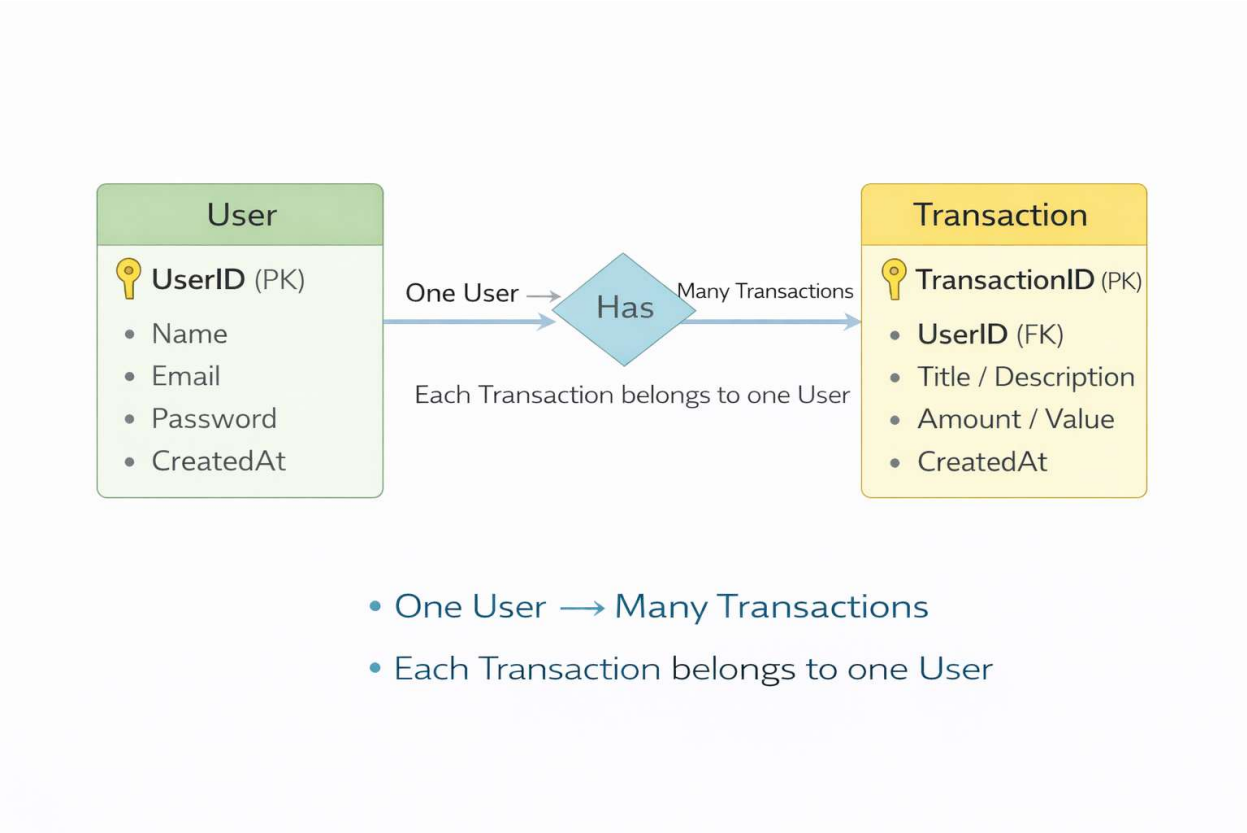
Backend Architecture (Node.js + Express)

- **Controllers:** Handle business logic for authentication and CRUD
- **Models:** Mongoose schemas for User and Transaction
- **Routes:** API endpoints for auth and CRUD operations
- **Middleware:** JWT verification and request protection
- **Configuration:** Database connection and environment variables

Database Architecture (MongoDB)

- Users Collection:
  `_id, name, email, password (hashed), createdAt`
- Transactions Collection:
  `_id, userId (FK), title/description, amount/value, createdAt`
- Relationship:
  Many Transactions belong to One User

ER Diagram



Features

Feature	Description
User Authentication	JWT-based registration and login
CRUD Operations	Create, Read, Update, Delete transactions
Protected Routes	Secure APIs using JWT middleware

MongoDB Integration	Store and fetch data using Mongoose

Roles and Responsibilities

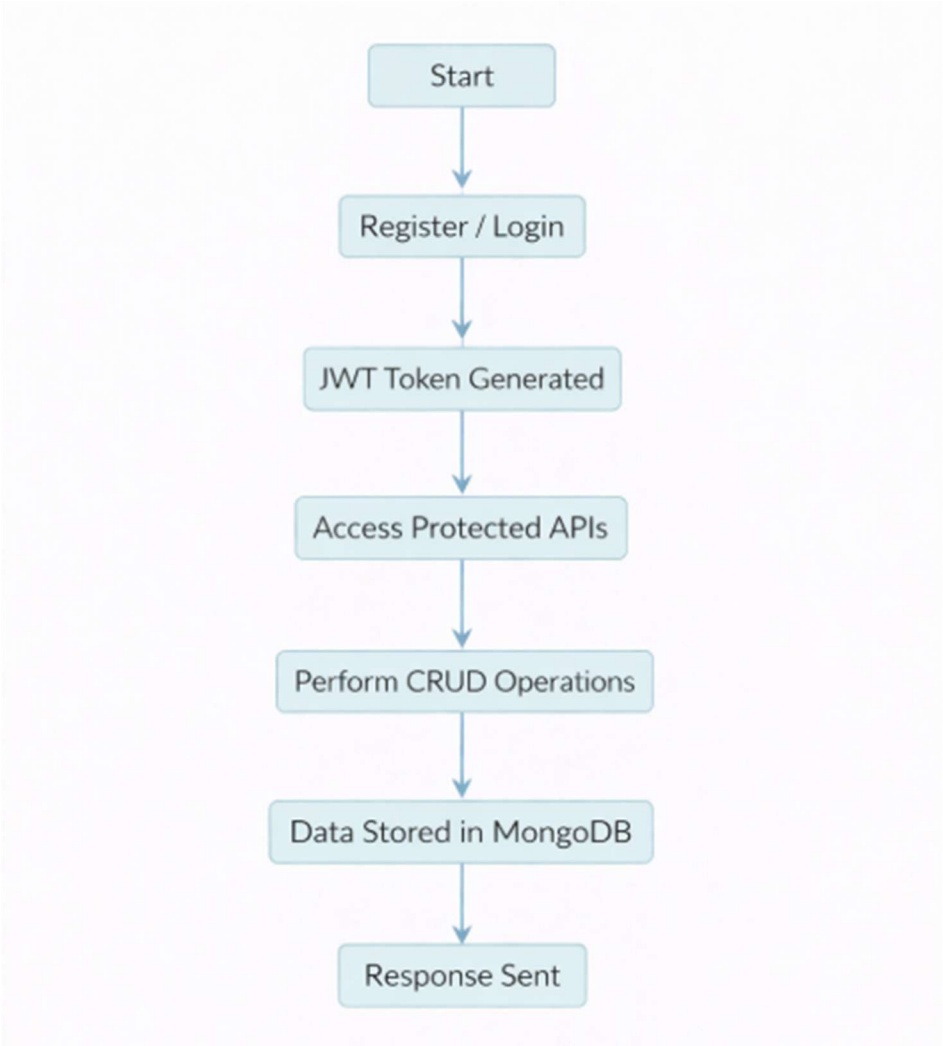
Student / Developer:

- Write backend code
- Create APIs
- Test APIs using Postman/Thunder Client
- Push code to GitHub

System (Backend):

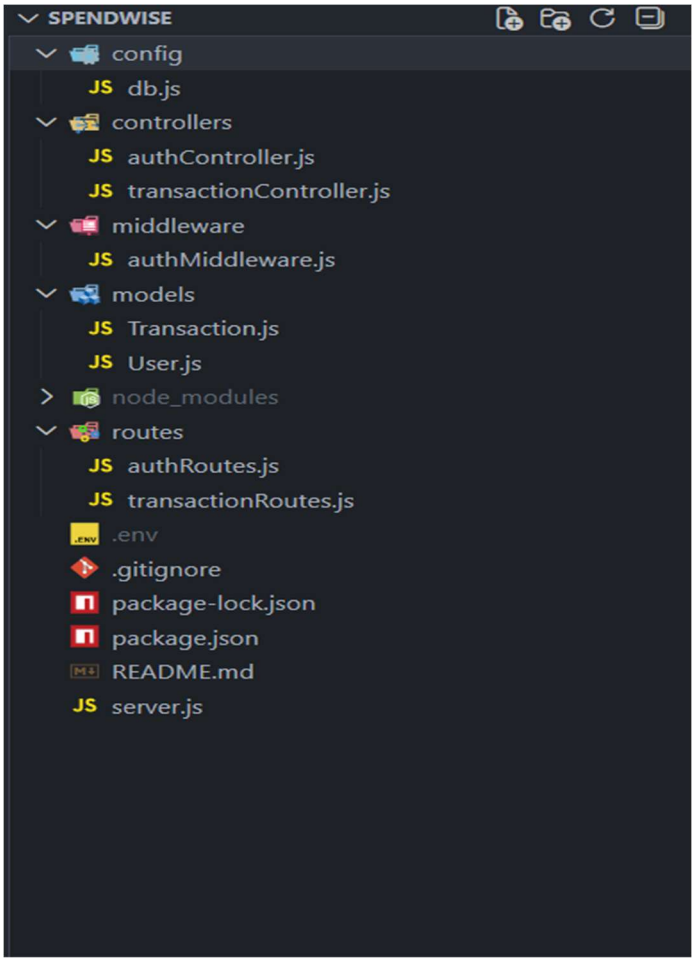
- Authenticate users using JWT
- Handle CRUD operations
- Store and fetch data from MongoDB
- Protect private routes

Users Flow:


Start
Register / Login
JWT Token Generated
Access Protected APIs
Perform CRUD Operations
Data Stored in MongoDB
Response Sent

PROJECT SETUP AND CONFIGURATION

Folder Structure



Project Setup

1. Install required tools:

    - Node.js
    - MongoDB

2. Create project folders:
    - server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:
    - Express
    - Mongoose
    - Cors

- Jsonwebtoken
- Bcrypt
- Dotenv

BACKEND DEVELOPMENT

- Setup Express server
- Create `server.js` file
- Create `.env` file and define PORT and DB URL

```
.env
PORT=5000
MONGO_URI=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=your_secret_key
```

- Configure middleware (cors, express.json)
- User Authentication:
  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware

- Define API Routes:
  - Auth routes

```
const router = express.Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

  - Transaction CRUD routes

- Implement Data Models:

  - User schema

```
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: true,
      minlength: 6,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

- Transaction schema

```
const transactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    amount: {
      type: Number,
      required: true,
    },
    type: {
      type: String,
      enum: ["income", "expense"],
      required: true,
    },
    category: {
      type: String,
      default: "General",
    },
    note: {
      type: String,
    },
    date: {
      type: Date,
      default: Date.now,
    },
  },
  { timestamps: true }
);
```

- Implement CRUD operations
- Add basic error handling

DATABASE DEVELOPMENT

Configure MongoDB

- Use MongoDB Atlas or local MongoDB
- Store connection URL in `.env` file
- Connect using Mongoose

Create Database Connection

- Create a config file for DB connection

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
config > JS db.js > ...
const mongoose = require("mongoose");

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error("MongoDB connection error:", error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

- Use `mongoose.connect()`
- Handle success and error cases

Create Schema and Models

User Model:

- Name
- email
- password

Data / Transaction Model:

- userId
- title / description
- amount / value
- timestamps

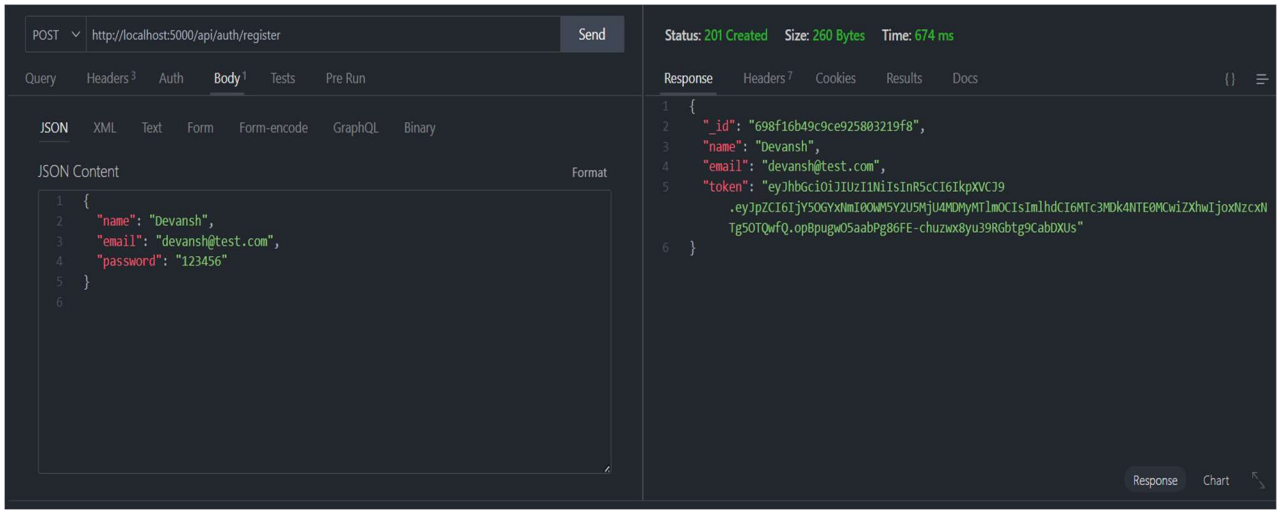
PROJECT EXECUTION

Project Execution Steps:

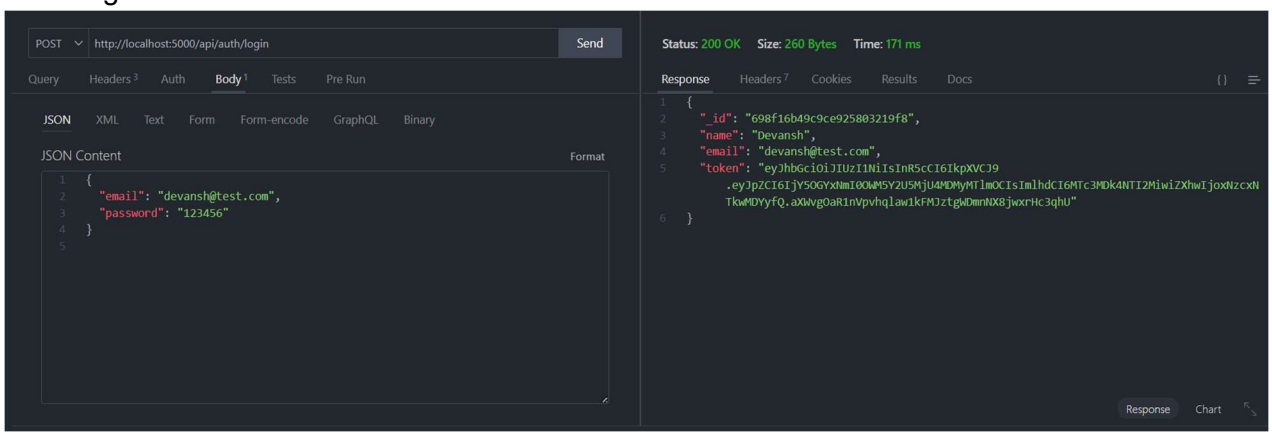
1. Start MongoDB
2. Run backend server

```
PS C:\Users\Devansh\Desktop\spendwise> npm run dev

> spendwise@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: ac-sg1nthw-shard-00-01.s3mna8f.mongodb.net
```

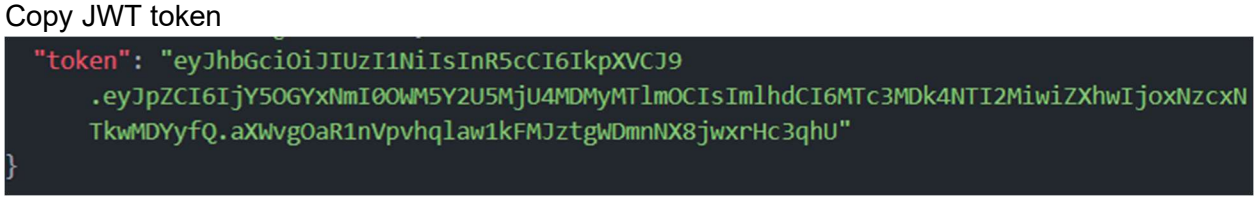
3. Test Register API



4. Test Login API



5. Copy JWT token



6. Use token to access protected CRUD APIs
7. Perform Create, Read, Update, Delete operations

OUTPUT SCREENSHOT

1. Register API

POST http://localhost:5000/api/auth/register Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Devansh",
  "email": "devansh@test.com",
  "password": "123456"
}
```

Status: 201 Created Size: 260 Bytes Time: 674 ms

Response Headers 7 Cookies Results Docs


```
{
  "_id": "698f16b49c9ce925803219f8",
  "name": "Devansh",
  "email": "devansh@test.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5OGYxNmI0OWM5Y2U5MjU4MDMyMTlmOCIsImlhdCI6MTc3MDk4NTE0MCwiZXhwIjoxNzcxN
      Tg5OTQwfQ.opBpugwO5aabPg86FE-chuzwx8yu39RGbtg9CabDXUs"
}
```


Response Chart

2. Login API

POST http://localhost:5000/api/auth/login Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "email": "devansh@test.com",
  "password": "123456"
}
```

Status: 200 OK Size: 260 Bytes Time: 171 ms

Response Headers 7 Cookies Results Docs


```
{
  "_id": "698f16b49c9ce925803219f8",
  "name": "Devansh",
  "email": "devansh@test.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5OGYxNmI0OWM5Y2U5MjU4MDMyMTlmOCIsImlhdCI6MTc3MDk4NTI2MiwiZXhwIjoxNzcxN
      TkwMDYyfQ.aXWvgOaR1nVpvhqlaw1kFMJztgWDmnNX8jwxrHc3qhU"
}
```


Response Chart

3. Create Transaction

POST http://localhost:5000/api/transactions Send

Query Headers 4 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "amount": 500,
  "type": "expense",
  "category": "Food",
  "note": "Lunch"
}
```

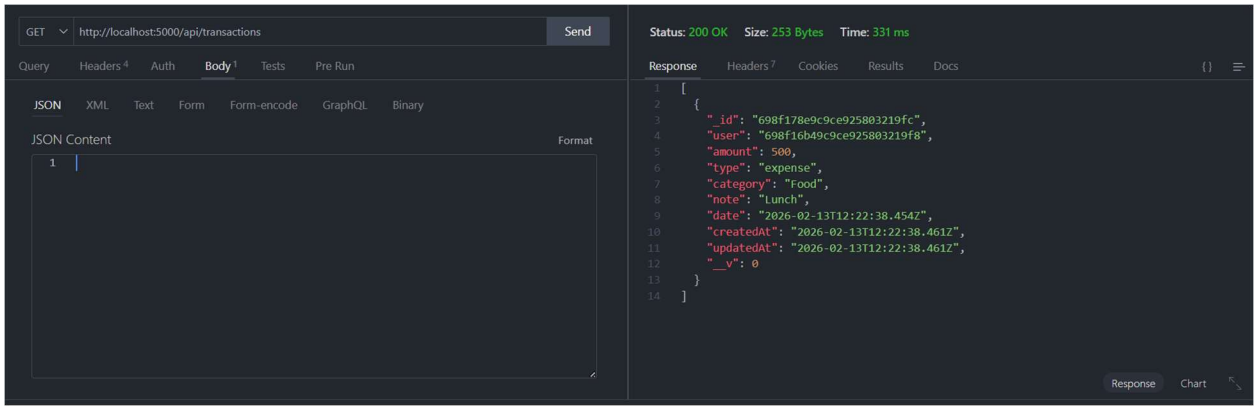
Status: 201 Created Size: 251 Bytes Time: 178 ms

Response Headers 7 Cookies Results Docs

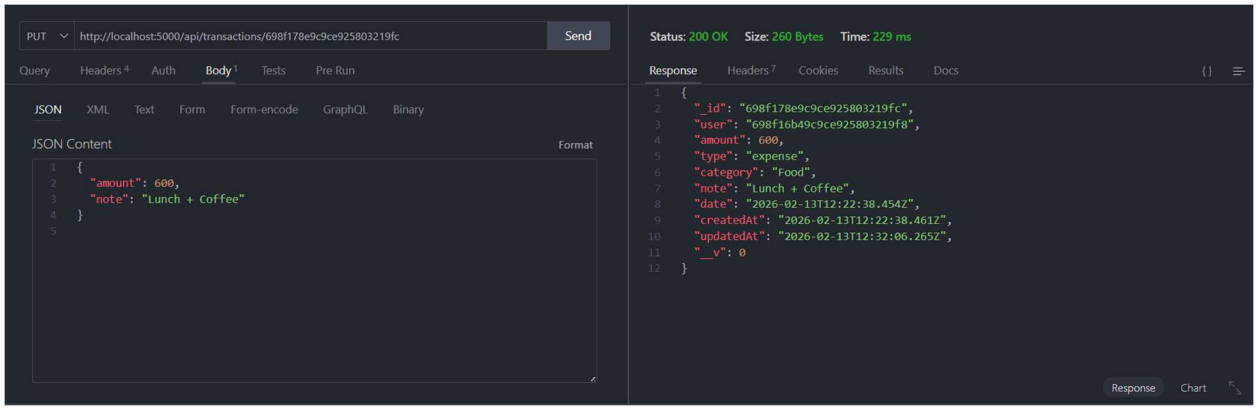
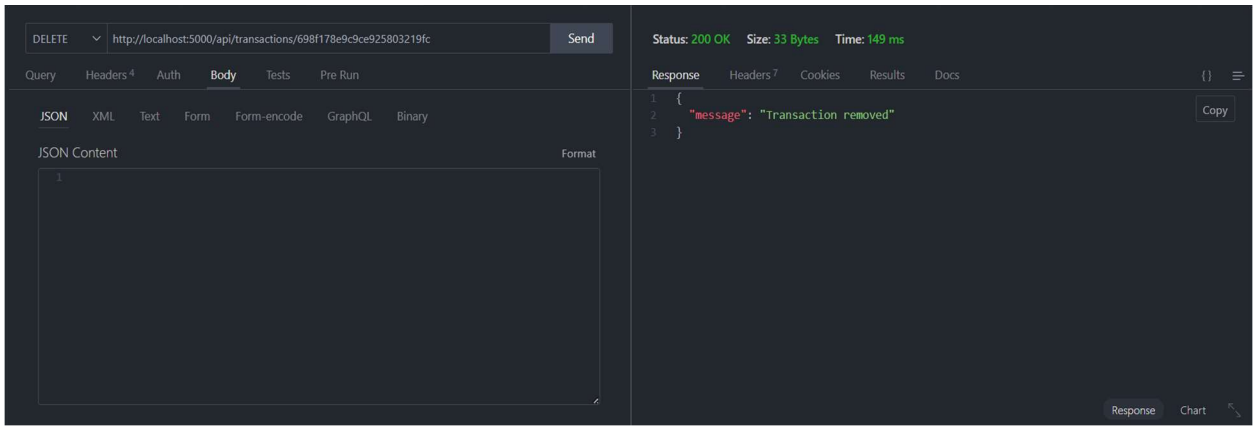

```
{
  "user": "698f16b49c9ce925803219f8",
  "amount": 500,
  "type": "expense",
  "category": "Food",
  "note": "Lunch",
  "date": "2026-02-13T12:22:38.454Z",
  "_id": "698f178e9c9ce925803219fc",
  "createdAt": "2026-02-13T12:22:38.461Z",
  "updatedAt": "2026-02-13T12:22:38.461Z",
  "__v": 0
}
```


Response Chart

4. Get information



5. Update API



6. Delete API